

Hardware Project

Wrist-camera SLAM for manipulation demonstrations and live teleoperation

28 August 2026

What the project is

A person picks up a handheld UMI-style gripper and performs a task by hand. Two things are recorded at once: what the wrist-mounted camera sees, and how far the jaws are open. Both are timestamped on one host clock and matched by nearest timestamp, with the per-frame error written out rather than assumed.

The camera pose is not a nice-to-have. It **is** the action label — the exported `robot0_eef_pos` and `robot0_eef_rot_axis_angle` are the camera trajectory. Bad tracking does not degrade the dataset; it invalidates it.

Component	Part
Camera	Intel RealSense D435i (stereo IR + RGB + IMU)
Actuator	Dynamixel XL330-M077 via ROBOTIS U2D2, driven as a torsional spring
Tracker	ORB-SLAM3 Stereo-Inertial, embedded via a C ABI shim
Arm	UFACTORY xArm over Ethernet, servo mode (teleoperation)
Output	Zarr replay buffer, UMI / Diffusion Policy schema
Code	~6,700 lines Python plus local ORB-SLAM3 changes

Two branches share one pose estimator: **capture → track → export → train** for data collection, and **capture → track → arm** for teleoperation. Anything that improves tracking improves both.

Problems and fixes

#	Problem	Fix	Status
1	Monocular VIO drifted without bound	Switched to ORB-SLAM3 Stereo-Inertial	Resolved

#	Problem	Fix	Status
2	USB latency made the servo spring unstable	Moved the PID onboard the servo	Resolved
3	Careful manipulation never cleared the IMU-init gate	Warm-up until IMU BA2, then start the demo	Verified on rig
4	The depth projector was being used as scene texture	Emitter off unless depth is running	Verified on rig
5	The gripper ate a fifth of the feature budget	Derived polygon mask	Measured
6	A held object becomes camera-fixed too	Deliberately not fixed in code	Open
7	Resets move the map origin mid-recording	map_epoch counter, export splits on it	Handled
8	A SLAM pose is not a controller pose	Clutch + three independent guards	Handled
9	Camera frame is not 90° from the robot frame	Full rotation fitted by Kabsch/SVD	Resolved
10	A loaded atlas is never actually entered	Abandoned — the warm-up supplies the same flag	Measured, unresolved
11	Merges and loop closures move the origin with nothing flagging it	Re-resolve against the final map; split on impossible speed	Handled
12	The warm-up is not the demonstration	Mark it with a keypress; drop it on export	Verified on rig

1-2. Fixed early

Monocular visual-inertial odometry had no absolute scale and drifted without bound. Stereo-inertial fixed it: the baseline supplies metric scale directly.

The servo was originally driven as a spring by a host-side PID loop over USB, where jitter in the control interval made the restoring force inconsistent. The loop now runs on the servo's own microcontroller; the host only sets the target.

3. Careful manipulation never cleared the IMU-init gate

Tracking failed during exactly the task the rig exists for: small-motion pick-and-place. Near-constant resets, ~33% of frames tracked, tracked runs about three frames long.

The cause is in ORB-SLAM3's `LocalMapping.cc`: IMU initialization demands at least 2 cm of accumulated translation within roughly ten seconds, or it resets. Precise manipulation has small net displacement by nature.

Fix, as designed — build the map once. A deliberate expansive scan is saved as `atlas.osa`; every demo loads it and localizes against it. The reset is gated on `!GetInertialBA2()`, `Map::serialize` writes that flag, so a loaded BA2-complete map should never re-trigger it. This turned out not to work — see problem 10.

```
python3 output_script/build_atlas_map.py --output-dir
maps/kitchen_table
python3 output_script/synced_capture.py --orb slam-map-dir
maps/kitchen_table
```

What the rig then showed. A live session still reset 18 times. Texture was ruled out on the spot — median 800 features per frame, only 4% of frames below 300. The IMU log gave the real answer:

Quantity	Median	p90
Angular rate	8.4 °/s	35 °/s
Linear acceleration, gravity removed	0.18 m/s²	1.03 m/s ²

The camera was being rotated, not translated. The gate measures distance between camera *centres*; pivoting in place changes the image enormously and contributes nothing. At this camera's intrinsics, 8.4 °/s of rotation produces ~79 px/s of image motion while 5 cm/s of translation produces only ~21 px/s — so roughly four fifths of the apparent motion carried no translation information.

The operating instruction: **carry the camera bodily sideways and back and forth, 20-30 cm, pointed at roughly the same scene.** Not standing still and sweeping it around like a torch. The same applies to the mapping pass.

What actually fixed it. The warm-up at the start of each recording completes IMU BA2 in the session's own map — "start VIBA 2 / end VIBA 2" in the log, about 20 s in — and from that point the gate stops applying for the rest of the recording. That is exactly what loading an atlas was supposed to provide, earned in-session instead of borrowed. scan_0014, a real pick-and-place recorded this way: the warm-up segment tracked 99% with one reset, and the demonstration segment that followed tracked 92% with none — against 33-35% and 66-100 resets for the same kind of task with no warm-up. Replayed offline the whole episode came out 1873/1905 tracked with zero resets.

The cost is that the warm-up is roughly 40 s of sweeping that is not the demonstration, which is problem 12.

4. The depth projector was being used as scene texture

The D435i's IR projector paints dots so stereo depth has something to match on blank surfaces. But the pattern is projected *from the camera*: the dots slide across surfaces as the camera moves, and they are near-identical to one another, so their descriptors are ambiguous. As landmarks they are worse than useless. The option had never been set, so it defaulted on, and a code comment had come to describe the dots as "the only texture a plain surface had to offer".

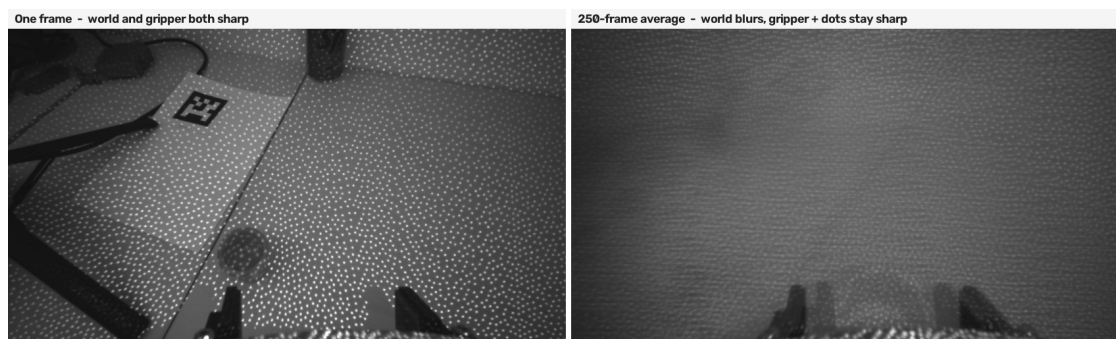


Figure 1. Left: one IR frame — dots, gripper and world all equally sharp. Right: 250 frames of the same recording averaged. Everything attached to the world smears into a gradient; the gripper silhouette and the dot pattern survive intact, because both are attached to the camera. This one image is the evidence behind problems 4 and 5.

Fix. Emitter on only when the depth stream is actually running. Derived from the depth stream rather than exposed as its own flag — no configuration wants a third answer. Upstream draws the same line: ORB-SLAM3's Stereo,

Stereo-Inertial, Monocular and Calibration RealSense examples all switch it off; only the RGB-D ones, which consume the depth map rather than IR features, leave it on. Confirmed running on the rig.

5. The gripper ate a fifth of the feature budget

The camera is wrist-mounted, so the gripper is rigidly attached and permanently in frame. A feature that never moves in the image is, geometrically, evidence that the camera never moved — so gripper features bias translation *downward*, in the one regime already closest to the 2 cm threshold. They also triangulate into bogus map points that get baked into the atlas.

Recording	Features / frame	On the gripper	Share
scan_0006	779	130	16.7%
scan_0007	795	146	18.3%
scan_0009	767	163	21.3%

The gripper occupies **8.4%** of the frame but takes **17-21%** of the features: a dark, hard-edged silhouette is exactly what ORB looks for, in a scene otherwise dominated by plain white surfaces.

Fix. A polygon mask flattened into the frames before tracking sees them. Gripper features drop to zero, the total is unchanged (779 → 778), so the budget goes to real scene content. The saved IR PNGs stay raw.

The polygon was derived, not hand-drawn: camera-fixed structure stays sharp in a long temporal average, so gradient magnitude of that average isolates it. Fitted by voting across twelve samples from three recordings, keeping only blobs touching the bottom edge, then dilated for finger travel. Extent came out stable to $\pm 0.1\%$ of frame area. Validated on a recording held out of the fit.

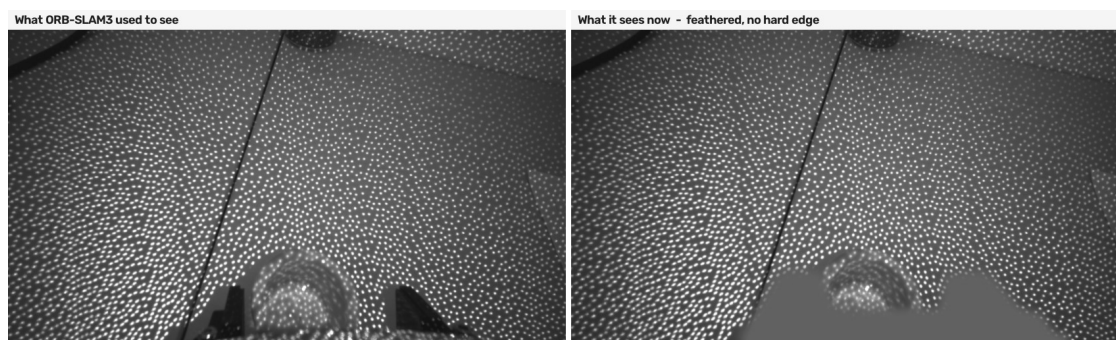


Figure 2. The same frame before and after masking. The fill is feathered rather than hard-edged, so the mask's own border does not become a new

set of camera-fixed corners; measured, the boundary band ends up with 12–20% fewer features after masking, not more.

Masking happens in Python because upstream cannot do it: `ORBextractor::operator()` takes a mask argument and ignores it, and `TrackStereo` has no mask parameter at all.

6. A held object becomes camera-fixed too — open

The moment an object is grasped it is rigidly attached to the camera, exactly like the gripper. A static mask cannot cover it, because where it lands depends on how it was grasped.

Recording	Grasp window	Object area	Share of post-mask features
scan_0009	12–20 s	2.8%	13.2%
scan_0007	6–12 s	2.3%	11.4%

Red = static gripper mask. Cyan = only camera-fixed while the object is held.

scan_0009, 12–20s

scan_0007, 6–12s

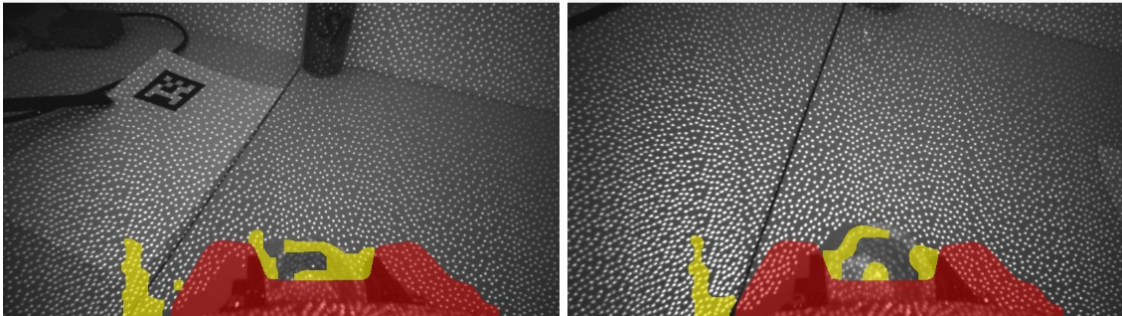


Figure 3. The camera-fixed detector run over short sliding windows instead of a whole recording. Red is the static gripper mask. Cyan is the region that is camera-fixed only while the object is held — it appears above the mask boundary, disappears on release, and no static mask can predict where it will land.

Position taken — deliberately not fixed in code. `PoseOptimization` already uses a Huber kernel and four rounds of chi-squared outlier rejection, which is exactly the mechanism for a minority of points that disagree with the rest, and 11–13% is a clear minority. Per-frame dynamic rejection would re-implement that with more ways to get it wrong.

Three reasons it is not settled: the share could multiply with a larger object or a blanker background; the object was mapped in its pre-grasp position, so its stale map points pollute the *atlas*, which outlives any single frame; and it happens precisely during the phase already nearest the IMU-init gate. The real lever is the denominator — more world-fixed texture around the workspace dilutes the object’s share. Extending the mask upward by 40–60

px is documented and deferred until there is live grasp-phase data to decide against.

7. Resets move the map origin mid-recording

Each reset can move the active map's pose origin, silently breaking any trajectory spanning it. A map-ID check does not catch it: the common reset path clears and re-initializes the same Map object in place, so its ID never changes. One recording showed two real resets that an ID check reported as zero.

Fix. A counter in Atlas, incremented on both `CreateNewMap()` and `clearMap()` so it catches every path, surfaced as `System::GetMapEpoch()` and written per row as `map_epoch`. Export splits on it: an episode is one contiguous map epoch, not one recorded folder.

Teleoperation

`output_script/umi_teleop.py` sends the same live pose to a UFACTORY xArm at 50 Hz through `set_servo_cartesian` in servo mode. Nothing on the arm is sensed; the hand is the input device. Anchor, clutch, deadzone, smoothing and delta clamp are copied from an existing Quest-controller script already tuned against this arm, so the one thing changed is the pose source. **Dry run is the default** — without `--execute` nothing reaches the arm.

This is also the harshest test of tracking quality available. Data collection judges a pose offline, where a bad trajectory is a bad label found later. Teleoperation judges it while it is being produced, and a bad pose is a moving arm.



Figure 4. Four IR frames from a teleoperation session — the world the tracker actually had to work with: a black monitor, a white wall, cabling. No projected dot pattern anywhere, the emitter change running live. The gripper sits at the bottom of every frame, the reason the mask exists.

8. A SLAM pose is not a controller pose

A Quest controller never loses tracking, reports at a fixed rate, and never redefines its own origin. ORB-SLAM3 does all three.

The SLAM worker is throttled to 10 Hz — 20 Hz reproduced a crash — while servo mode wants 50 Hz, so the control loop runs at 50 Hz and reads whatever the latest pose is. The arm is always chasing a target up to ~100 ms old. That is inherent, not a tuning defect.

A reset *moves the map origin*: the same pose value suddenly means somewhere else, which reaches a real arm as a jump. Tracking lost, a changed map_epoch, or no fresh pose within 0.5 s all disengage the clutch and demand a re-anchor. Three further guards bound what can get through:

Guard	Value	Reasoning
MAX_CART_DELTA_MM	80 mm	bounds distance from the anchor
POSE_JUMP_MPS	2.0 m/s	a hand carrying a camera never exceeds this; a SLAM jump is metres instantly
MAX_STEP_MM	8 mm / cycle	caps the arm at ~0.4 m/s whatever the cause

The distance clamp alone is not enough: at TARGET_ALPHA 0.55 an 80 mm step in the target still puts 44 mm into the very next 20 ms command, about 2.2 m/s of commanded motion.

Scale is 300 mm of arm per metre of hand, half the Quest script's 600. A SLAM pose carries tracking noise a controller pose does not, and scaling down attenuates it by the same factor. The extra hand travel is absorbed by the clutch — disengage, reposition, re-engage, like lifting a mouse.

Servo mode also does no reachability check; it accepts or rejects. Rejections are now named rather than printed as a bare return code, and recovered from automatically, so one rejection disengages the clutch instead of ending the session. Error 23, joint angle exceeds limit, is not something software can tune away — it means starting nearer the middle of the joints' range.

9. The camera frame is not a 90° rotation from the robot frame

How the camera frame relates to the arm's base frame depends on how the gripper is held and how the arm is mounted, so it is calibrated once by moving along each robot axis and reading which camera axis responded. The result is sanity-checked by determinant: a real rigid mounting can only give a proper rotation, so a single flipped sign gives -1 and is rejected on the spot rather than at --execute time.

An axis map was enough for translation. Rotation broke immediately:

ORB-SLAM3's world frame has its **Z aligned to gravity** by the IMU, but its **yaw is set by wherever the camera pointed when the map initialized** — arbitrary, and different every session.

An axis permutation only expresses multiples of 90°, so snapping an arbitrary yaw to the nearest axis leaves up to 45° of residual. Translation survives that as a cosine factor, which is why position-only teleoperation felt correct. Rotation does not: the whole rotation *axis* ends up tilted, and the wrist turns about a visibly wrong axis.

Fix. The three calibration directions already over-determine the full rotation; snapping them to axes threw that away. They are now fitted with Kabsch/SVD, determinant forced to $+1$. Against a synthetic 33° yaw offset with hand noise included:

Method	Angular error
Axis-map permutation	33.00°
Full fitted rotation	0.75°

This has a consequence for the atlas. Because the world yaw is chosen afresh whenever a map cold-starts, a fitted frame is only valid for the session

that produced it. Loading a saved atlas fixes the world frame and makes the calibration reusable — a third distinct job for the atlas:

	What the atlas was expected to buy — none of it delivered
1	Bypasses the 2 cm IMU-initialization gate — the warm-up does this instead
2	Relocalization instead of restarting when tracking is lost — never observed
3	Fixes the world yaw, so a frame calibration survives across sessions — still missing

Rotation stays behind --rotation, damped by a 0.5 scale, a 2° deadzone (subtracted rather than thresholded, so crossing it is continuous) and heavier smoothing than translation. The camera is held in the hand, where wrist articulation produces far more rotation per unit of effort than whole-arm translation does, and 1:1 feels twitchy. The arm's Euler convention was measured rather than read from the docs: only $R_z(\text{yaw}) \cdot R_y(\text{pitch}) \cdot R_x(\text{roll})$ reproduced the axis-angle rotation, to 0.000°.

The data-collection pipeline

10. A loaded atlas is never actually entered — unresolved

Loading an atlas does not put tracking into it. LoadAtlas restores the saved map, then the session cold-starts its own ("Creation of new map with id: 1") and tracks there until LoopClosing recognises the region and Atlas::ChangeMap switches over. The IMU-init gate reads `mpCurrentKeyFrame->GetMap()->GetInertialBA2()` — the current map, the fresh one, whose flag is false. The bypass only ever applies after a merge.

That merge has never been observed. Every recording replayed with --map-dir reports "atlas merge NEVER", including against an atlas built in the same scene fourteen minutes earlier. It is not the entry conditions either: the three checks `LoopClosing::NewDetectCommonRegions` returns early on all open at 21.6 s on scan_0013 and stay open, keyframes steady at 110–130, place recognition running thirty further seconds without a single match. Why it never matches is open.

Position taken — abandoned rather than pursued. The warm-up supplies the same flag, and the pipeline is verified without an atlas. What is still missing is a world frame that survives across sessions: ORB-SLAM3's yaw is set by wherever the camera pointed at map initialization, so a camera-to-robot

calibration is valid only for the session that produced it and trajectories from different recordings cannot share a frame. That blocks rotation teleoperation and combining recordings; it does not block collecting a dataset.

11. Merges and loop closures move the origin with nothing flagging it

map_epoch catches resets. It cannot catch a rigid correction applied inside one map — a loop closure or bundle adjustment moves every keyframe, and neither of the counter's increment sites is on that path. Nothing else in the row changes either: is_lost stays 0, timestamps stay continuous, the RGB match stays exact. Every column reads healthy because every column is healthy; what moved is which coordinate frame the numbers are in.

In scan_0011, inside one map_epoch segment: median frame-to-frame motion 1.4 cm, with steps of 23.3, 38.2 and 40.4 cm — 7.0, 10.9 and 4.0 m/s. A hand carrying a camera does not reach 10.9 m/s. Those were going into robot0_eef_pos_delta at about 30x the median, corrupting 7.9% of 16-frame training windows.

Fix, in two layers. Each episode is re-resolved against its final map via SaveTrajectoryTUM, which absorbs the discontinuity rather than recording it. And the exporter splits wherever consecutive poses imply more than 2.0 m/s — umi_teleop.py's POSE_JUMP_MPS, the same number on purpose, since it is one claim about the same hardware. Splitting, not smoothing: poses on both sides are individually correct in their own frame.

12. The warm-up is not the demonstration

A recording opens with sweeping motion that exists only to initialize tracking. It has to be recorded — offline replay has nothing else to work from — and must not be trained on. On scan_0014 it was 1370 of 1905 frames, 72%, and it passes every check in the exporter: those frames really are well tracked, well synced and correctly labelled. They are simply not the demonstration, which is not a property of the data.

Fix. A keypress during recording marks where the demo starts; the exporter drops everything before it. No heuristic — first gripper closure is the tempting one and it is wrong, since approaching with the gripper open is part of the demonstration. UMI does not guess either: its 06_generate_dataset_plan.py filters on a check_result.txt a human wrote after watching the video. The offset doubles as a measurement of warm-up cost, currently about 40 s.

Design note: the added texture is not a marker system

Stickers go *around* the manipulation region, and are never detected as markers. With the projector off, the workspace has to supply its own world-fixed texture, and printed patterns do that — but they are consumed as **ordinary ORB features**. There is deliberately no fiducial-detection code in the repository.

Deriving pose from marker geometry would tie the recorded trajectories to an instrumented environment, and a policy trained on those poses would depend on markers existing at deployment. Implicit use keeps the pose coming from natural features — the same estimator that runs when the markers are gone.

Two practical constraints: tracking happens in near-infrared, where many coloured dyes are transparent, so patterns must be black-on-white laser print and non-repeating; and whatever texture is present during mapping is baked into the atlas as map points, so it must stay physically put afterwards.

Where it stands

Item	Evidence	Status
OpenVINS to ORB-SLAM3	Live, 50+ s continuous motion	Verified
Onboard servo PID	Live, including hard nudges	Verified
Emitter switches off on the rig	Live session log and frames	Verified
Axis calibration	Live, determinant check passed	Verified
Teleoperation, translation only	Live, arm driven from SLAM pose	Verified
Rotation, not translation, caused the resets	Live IMU log	Measured
Gripper feature share	Offline, three recordings	Measured
Held-object feature share	Offline, two grasp windows	Measured
Fitted frame beats the axis map	Synthetic 33° yaw	Measured
Two-stage atlas	Live, 5 recordings, and	Tested — does not

Item	Evidence	Status
removes resets	against a same-scene atlas	work
Mask improves tracking	Feature counts only, not tracking	Needs rig
Teleoperation with rotation	Written, not yet driven	Needs rig
Jump and rate guards fire correctly	Never triggered live	Needs rig
Warm-up to IMU BA2 removes resets	Live, scan_0014: 92% and 0 resets through the task	Verified
Offline tracking beats live	scan_0011 both ways: 2.8x the training windows	Verified
Record → replay → export, end to end	scan_0014 → 535 frames, 1 episode, no splits	Verified
Warm-up marking drops the right frames	scan_0014: 1370 of 1905 frames dropped at the mark	Verified

Data collection now runs end to end without an atlas: record with a warm-up, mark where the demonstration starts, track offline at full frame rate, export. Verified on scan_0014 — 535 frames, one episode, no coordinate-frame splits, largest step 2.0 cm. Translation-only teleoperation runs on the real arm. What remains unvalidated is the mask's effect on tracking (feature counts only), rotation teleoperation, and the teleoperation guards, all of which need the rig.

Order to test in. The atlas steps are gone; what is left is coupled to scene texture, which still has to exist before any of it is worth measuring:

1. Put the scene texture up — around the manipulation region, not in it.
2. Record demos: warm up until the monitor turns green, press m, do the task.
3. Replay and export; check tracked ratio, resets and demo_start_offset_s per episode.
4. Record a control with --no-gripper-mask and compare tracking, not feature counts.
5. Re-run the frame calibration, then teleoperate with --rotation — dry run first.

Still open. Whether to mask inside ORB-SLAM3 rather than in Python — its octree extractor actively redistributes features toward sparse regions, which the cv2.ORB proxy used for the boundary measurement does not. And whether to extend the mask upward over where held objects sit, at the cost of the manipulation region's own pixels, which SLAM wants nothing from anyway. And why place recognition never matches against a loaded atlas — the entry conditions are open and it runs for tens of seconds without a hit, so the next step there would be instrumenting NewDetectCommonRegions to see whether merge candidates are returned at all.